Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas - Departamento de Ciencias de la Computación
CC4102: Diseño y Análisis de Algoritmos

Tarea 2: Árboles B y B^ε en memoria externa

Implementación y evaluación experimental de árboles B y Bε

Integrantes: Bastián Caballero, Julio Yáñez e Isidora Moncada

---

1 Introducción

En memoria externa, el costo de una operación se mide en transferencias de bloques entre el disco y la memoria principal (I/Os). El árbol B es la estructura clásica para un diccionario ordenado, en este cada nodo ocupa un bloque con $\Theta(B)$ llaves, por lo que insertar y buscar cuesta $\Theta(\log_B N)$ I/Os. El árbol B^ε disminuye particularmente el costo de inserciones, en este caso, cada nodo interno usa solo del orden de B^ε llaves divisorias y destina el resto del bloque a un buffer de inserciones pendientes que guarda los elementos que van llegando a diferencia del árbol B que las baja inmediatamente a sus hijos, cuando el buffer se llena, las inserciones pendientes son enviadas por lotes hacia los hijos. Así, cada I/O de un vaciado mueve muchos pares a la vez y una inserción cuesta $O(\log_B N/(\varepsilon B^{1-\varepsilon}))$ I/Os amortizadas, a cambio de una altura $O(\log_B N/\varepsilon)$, que es lo que pagan las búsquedas.

En esta tarea se implementa en C++ un árbol B ($b = 340$) y dos árboles B^ε, con nodos de 4096 bytes, en donde uno posee $\varepsilon \approx 0{,}25$ (c = 4, $l_{\max}$ = 506) y el otro $\varepsilon \approx 0{,}75$ (c = 126, $l_{\max}$ = 384). Con ellos se indexaron los primeros $N \in \{2^{15}, ..., 2^{26}\}$ pares medidos en la estación Quinta Normal entre 2019 y 2025. Los árboles se construyen sobre un arreglo de bloques en RAM que simula el disco y cuenta cada lectura y escritura; luego se guardan en un archivo binario y se hacen 50 búsquedas de rangos de una semana leyendo los nodos desde ese archivo. Se reportan el tiempo y las I/Os de la construcción, el tamaño y la altura de cada árbol, el tiempo y las I/Os de las búsquedas. Además se estudia el parámetro α del vaciado con $N = 2^{24}$.

En este informe también se describen las estructuras, los algoritmos y decisiones de implementación. Se presenta el entorno, la metodología y los resultados; para luego analizarlos con el análisis amortizado de los vaciados y la altura en función de ε, para finalmente concluir de acuerdo a la hipótesis inicial:

Hipótesis. Un bloque de 4096 bytes tiene espacio para $B = 512$ pares de 8 bytes, $B^{0{,}25} \approx 4{,}8$ y $B^{0{,}75} \approx 108$, que corresponden aproximadamente a los 5 y 127 hijos de las dos configuraciones.

1. *Inserción.* En un árbol B, la inserción implica recorrer el camino desde la raíz hasta una hoja y a modificar esa hoja: aproximadamente $h + 1$ I/Os por par, donde h está entre 2 y 4. En un árbol B^ε, cada par se deja primero en el buffer de la raíz (2 I/Os). Solo cuando el buffer se llena se envía un lote de pares a un hijo. Por lo anterior, el costo de ese vaciado se reparte entre todos los pares del lote. Con $\varepsilon \approx 0{,}25$, cada vaciado lleva al menos $\lceil 506/5 \rceil = 102$ pares al hijo más cargado; con $\varepsilon \approx 0{,}75$, lleva solo $\lceil 384/127 \rceil = 4$, por tanto se espera que B^ε con $\varepsilon \approx 0{,}25$ requiera menos I/Os por inserción que el de 0,75, y a su vez, ambos inferior que el árbol B.
2. *Altura y búsqueda.* La altura de un B^ε crece como $1/\varepsilon$: el árbol con $\varepsilon \approx 0{,}25$ tendría 3 veces la altura del de 0,75, que a su vez debería ser casi de la altura del árbol B. En búsquedas que tocan pocos nodos eso ordena los árboles como B $\approx B^\varepsilon$ (0,75) < B^ε (0,25). Pero un rango de una semana contiene $\approx 0{,}29$ % de los datos, así que para N grande debería dominar el número de bloques con resultados y los tres árboles deberían acercarse.
3. *Tamaño.* Las hojas de los B^ε tienen espacio para más pares que las del árbol B (506 y 384 contra 340), por lo que que ocuparían menos bloques, sobre todo con $c = 4$.

2 Desarrollo

2.1 Disco y archivo

Los nodos del disco simulado se reservan en segmentos de 1024 bloques (4 MiB) con `mmap`, incorporándolos a medida que el árbol crece. Así se evita reservar de inmediato la memoria del árbol más grande.

Las búsquedas se realizan con `O_DIRECT`, tras sincronizar el archivo y descartar sus páginas de la caché. Esto permite medir accesos al disco y no lecturas que el sistema operativo ya mantenga en RAM. En Btrfs se usa NOCOW y se verifica con `FIEMAP` que los bloques no estén comprimidos; si no se puede asegurar lectura directa, se registra la condición en los resultados.

2.2 Árbol B^ε

- Durante un vaciado, el nodo que lo inicia se conserva en memoria y se escribe una sola vez al finalizar. Esto evita escrituras intermedias al procesar sus lotes.
- Al dividir una hoja que recibe un lote, primero se mezclan sus pares con el lote y luego se parte el resultado por la mediana, de esta forma ambas hojas quedan dentro de su capacidad.
- Las llaves creadas por divisiones durante un vaciado se incorporan mientras el nodo padre está en memoria; antes de escribirlo, se divide por la mediana las veces necesarias. Con ello se mantiene la estructura válida sin escribir el padre después de cada división hija.
- El valor de α se representa como un entero escalado por 10^6 y el umbral se calcula con aritmética entera. Esto hace que $\lceil \alpha f \rceil$ sea reproducible y no dependa de redondeos de punto flotante.
- Las búsquedas tratan los intervalos de los hijos como cerrados al decidir cuáles visitar. Así se incluyen todos los pares cuando una llave repetida queda a ambos lados de una llave divisoria. El resultado se ordena antes de escribir el CSV para entregar las consultas en orden de llave.

2.3 Verificación

Cada búsqueda se compara con fuerza bruta sobre el mismo prefijo de datos y el programa termina con error ante una diferencia. Por otra parte, las pruebas automatizadas revisan invariantes de los árboles, conteos de I/O y casos con llaves repetidas y valores extremos, lo que permite detectar errores de estructura antes de analizar las mediciones.

3 Resultados

3.1 Entorno

Componente	Valor
Sistema operativo	Ubuntu / Linux kernel 7.0.0-38-generic (x86_64)
Procesador	AMD Ryzen 7 7840HS (8 núcleos, 16 hilos)
Cachés	L1d: 32 KiB × 8 (256 KiB); L1i: 32 KiB × 8 (256 KiB); L2: 1 MiB × 8 (8 MiB); L3: 16 MiB
RAM	14 GiB DDR5 (swap de 511 MiB)
Compilador	`g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic`
Informe	Typst 0.15.1

Tabla 1: Entorno de hardware y software en que se ejecutaron los experimentos.

3.2 Datos y metodología

`datos.bin` (512 MiB, con el sha256 publicado) contiene $2^{26} = 67\,108\,864$ pares en orden aleatorio, con llaves entre 1 546 300 802 (01-01-2019 00:00:02 UTC) y 1 754 006 394 (31-07-2025 23:59:54 UTC), 11 llaves repetidas y temperaturas entre −2,53 y 38,32 °C. Para cada N se usan los primeros N pares; como el archivo está desordenado, cada prefijo es una muestra aleatoria de todo el período y está contenido en los prefijos más grandes.

Experimento principal. Cada combinación (N, árbol) se construyó una vez (36 construcciones) y cada construcción se siguió de las mismas 50 búsquedas (1 800 en total, todas iguales a la fuerza bruta). El tiempo y las I/Os de búsqueda son el promedio de las 50 búsquedas. Ambos árboles B^ε usan $\alpha = 0{,}9$ (justificado en la Sección 3.7).

Bonus. Con $N = 2^{24}$, para el B^ε con $c = 126$ ($\varepsilon \approx 0{,}75$) y cada uno de los 8 valores de α (0,1; 0,25; 0,4; 0,5; 0,6; 0,75; 0,9; 0,99) se construyó el árbol y se hicieron las mismas 20 búsquedas (generadas con otra semilla).

Los tiempos dependen de la máquina y de su carga al medir; las I/Os, el tamaño y la altura no.

3.3 Creación

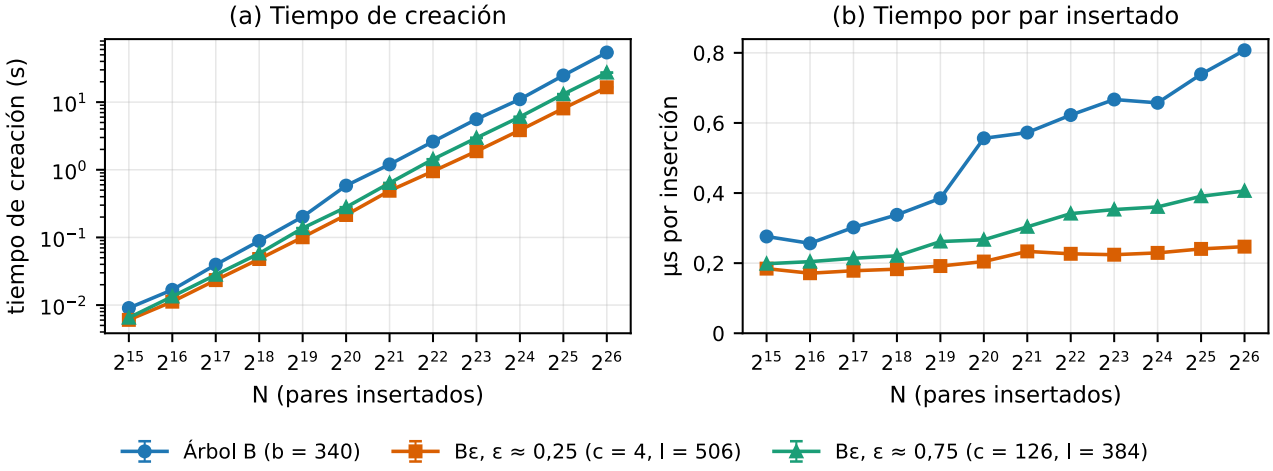


Figura 1: Tiempo de creación (inserción de los N pares) en función de N: (a) total; (b) por par insertado.

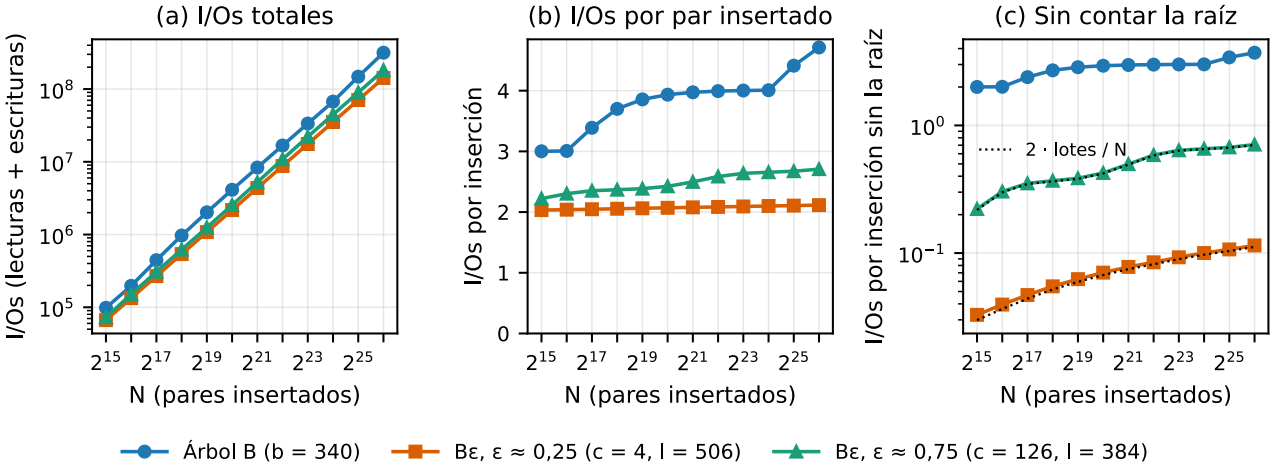


Figura 2: I/Os simuladas al insertar los N pares: (a) totales; (b) por par insertado; (c) por par sin contar la raíz (1 lectura en el árbol B, 1 lectura y 1 escritura en los B^ε). La línea punteada es $2 \cdot \text{lotes}\ / N$: leer y escribir el hijo en cada lote de un vaciado.

- El tiempo de creación crece casi linealmente con N (Figura 1 a). El orden B^ε $c = 4$ < B^ε $c = 126$ < B se cumple en 11 de los 12 valores de N (no en 2^{15}). Con $N = 2^{26}$: 16,6 s, 27,2 s y 54,2 s.
- Por par insertado (Figura 1 b), entre $N = 2^{15}$ y $N = 2^{26}$ el árbol B pasa de 0,27 a 0,81 µs, el B^ε $c = 126$ de 0,18 a 0,41 µs y el B^ε $c = 4$ de 0,18 a 0,25 µs.
- Las I/Os por inserción (Figura 2 b) pasan de 3,002 a 4,712 en el árbol B, de 2,221 a 2,707 en el B^ε $c = 126$ y de 2,033 a 2,115 en el B^ε $c = 4$. El orden B^ε $c = 4$ < B^ε $c = 126$ < B se cumple para todo N.
- Sin contar la raíz (Figura 2 c), con $N = 2^{26}$ el árbol B hace 3,712 I/Os por inserción, el B^ε $c = 126$ 0,707 y el B^ε $c = 4$ 0,115, es decir, 5,3 y 32,3 veces menos que el árbol B. Con $N = 2^{26}$, la línea $2 \cdot \text{lotes}\ / N$ queda a 0,7 % (c=126) y 2,8 % (c=4) de la curva medida.

Medida	B	B^ε c=4	B^ε c=126
Nodos internos	1 041	64 893	3 094
Hojas	278 430	139 200	250 234
% de los pares en nodos internos	0,41	29,87	1,14
Pares por hoja (promedio)	240,0	338,1	265,1
Vaciados	-	1 953 618	753 963
Pares por lote (promedio)	-	158,0	8,4
Bajadas por par	-	8,83	2,95
Lecturas por par insertado	3,700	1,056	1,352
Escrituras por par insertado	1,012	1,059	1,355
ms por I/O de búsqueda	0,308	0,403	0,377

Tabla 2: Estructura de los árboles y contadores de la inserción con $N = 2^{26}$. *Bajadas por par*: total de pares bajados un nivel (sumando todos los lotes) dividido por N. La última fila es el tiempo de búsqueda dividido por sus I/Os.

3.4 Tamaño y altura

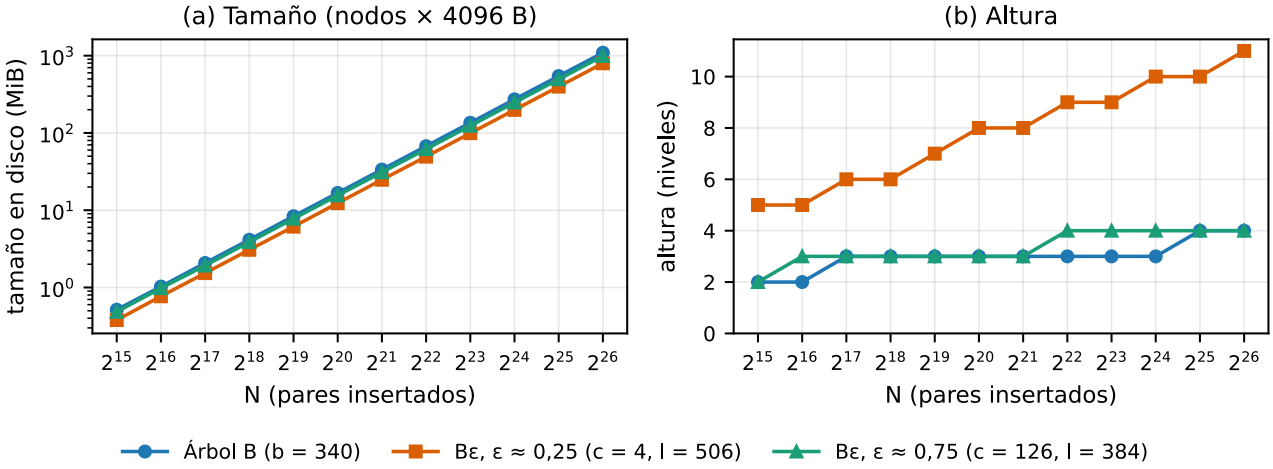


Figura 3: (a) Tamaño de los árboles (cantidad de nodos por 4096 bytes, que es el tamaño del archivo) y (b) altura, en función de N.

- El tamaño crece proporcionalmente a N (Figura 3 a). Con $N = 2^{26}$, el árbol B usa 279 471 nodos (1 091,7 MiB); el B^ε $c = 126$, 253 328 (989,6 MiB, 9 % menos), y el B^ε $c = 4$, 204 093 (797,2 MiB, 27 % menos). El orden es el mismo para todo N.
- La altura (Figura 3 b) del árbol B pasa de 2 a 4 (sube en 2^{17} y 2^{25}) y la del B^ε $c = 126$ de 2 a 4 (sube en 2^{16} y 2^{22}). La del B^ε $c = 4$ pasa de 5 a 11 (sube en 2^{17}, 2^{19}, 2^{20}, 2^{22}, 2^{24} y 2^{26}).
- Las hojas del árbol B tienen en promedio 240 pares (71 % de 340); las del B^ε $c = 126$, 265 (69 % de 384), y las del B^ε $c = 4$, 338 (67 % de 506). Además, el 29,9 % de los pares del B^ε $c = 4$ está en buffers de nodos internos, contra 1,1 % en el de $c = 126$ y 0,4 % en los nodos internos del árbol B (Tabla 2).

3.5 Búsquedas

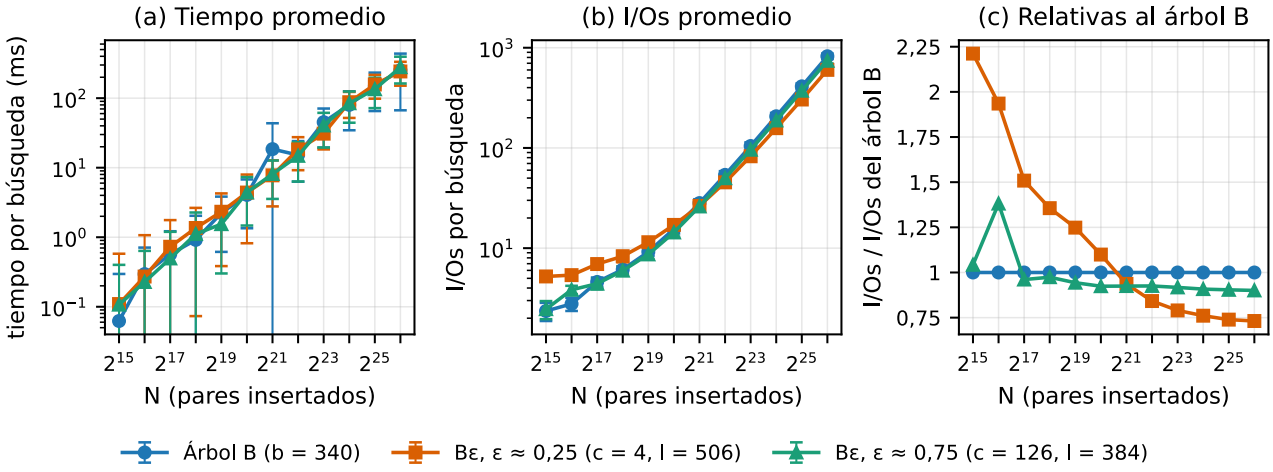


Figura 4: Búsquedas de rangos de una semana (promedio de las 50 búsquedas) en función de N: (a) tiempo por búsqueda y (b) I/Os por búsqueda, con barras de ± una desviación estándar; (c) I/Os de cada árbol divididas por las del árbol B.

- Cada búsqueda devolvió en promedio el 0,291 % de los N pares (lo esperado para rangos de una semana es 0,291 %), desde 97 pares con $N = 2^{15}$ hasta 195 420 con $N = 2^{26}$; los tres árboles devolvieron los mismos pares.
- Las I/Os por búsqueda (Figura 4 b) crecen casi proporcionalmente a N cuando N es grande. Con $N = 2^{15}$ el B^ε $c = 4$ lee 5,2 bloques, contra 2,4 del árbol B (2,17 veces) y 2,5 del B^ε $c = 126$. Desde $N = 2^{22}$, el B^ε $c = 4$ es el que menos lee (Figura 4 c): con $N = 2^{26}$ hace 601,5 I/Os, contra 740,7 del B^ε $c = 126$ y 822,4 del árbol B (0,73 y 0,90 veces las del árbol B).
- El tiempo de búsqueda (Figura 4 a) sigue a las I/Os: con $N = 2^{26}$, 242,2 ms (B^ε $c = 4$), 278,8 ms (B^ε $c = 126$) y 252,2 ms (B), entre 0,308 y 0,403 ms por I/O (Tabla 2). El B^ε $c = 4$ es el más lento de los tres con $N = 2^{17}$, 2^{18}, 2^{19}, 2^{22}, 2^{24} y 2^{25} y el más rápido desde $N = 2^{26}$. Entre el árbol B y el B^ε $c = 126$, con $N = 2^{26}$ la diferencia es de 26,6 ms (el B^ε $c = 126$ es el más lento). Como cada búsqueda se midió una sola vez, las diferencias de tiempo chicas pueden deberse a la variación del disco.

3.6 Una misma búsqueda con dos valores de N

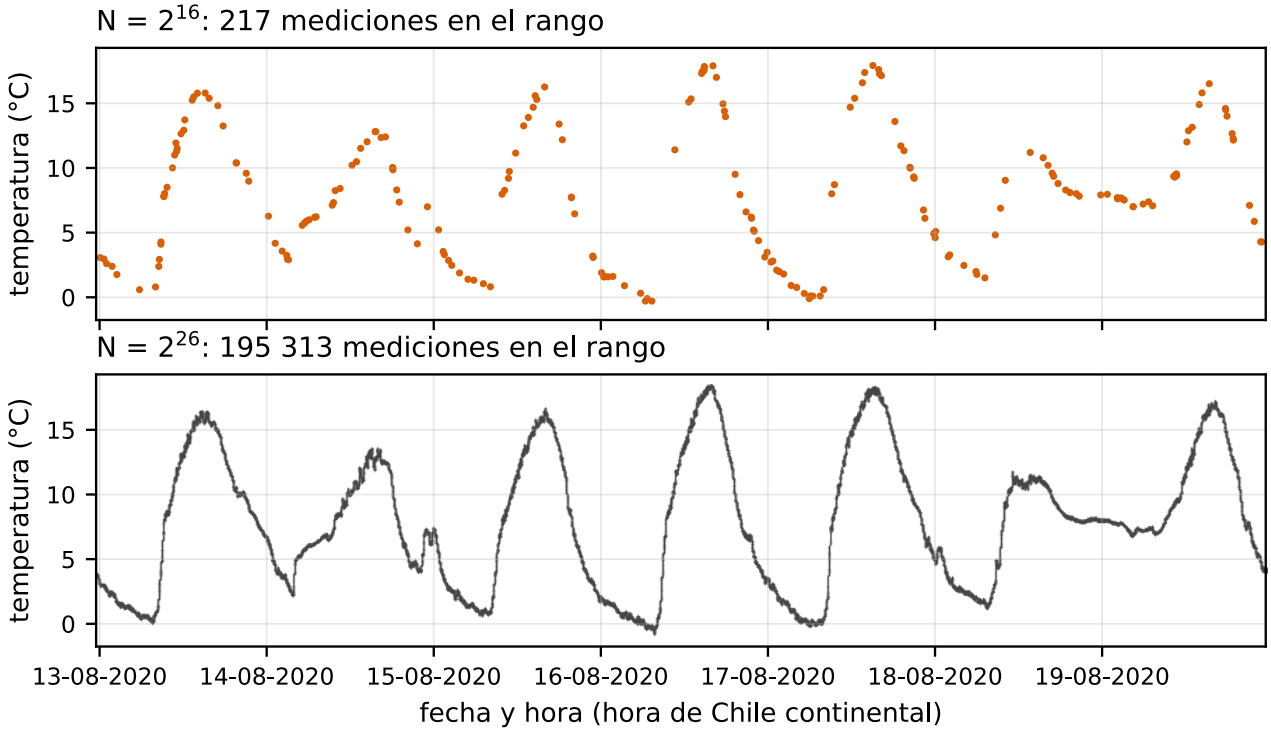


Figura 5: Pares que devuelve la búsqueda 0, $[1\,597\,289\,407, 1\,597\,894\,207]$ (del 12-08-2020 a las 23:30 al 19-08-2020 a las 23:30, hora de Chile), con $N = 2^{16}$ y 2^{26}.

N	Pares	% de N	Mín. (°C)	Máx. (°C)	Prom. (°C)	Separación mediana (s)	¿Contenido en el de 2^{26}?
2^{16}	217	0,331	-0,29	17,92	8,21	2 214	sí
2^{26}	195 313	0,291	-0,80	18,33	7,86	2	sí

Tabla 3: Resumen de la búsqueda 0 para varios valores de N. *Separación mediana*: mediana del tiempo entre dos mediciones consecutivas del resultado.

La búsqueda 0 cubre una semana (Figura 5). Con $N = 2^{16}$ devuelve 217 mediciones, separadas por 37 minutos en la mediana, y con $N = 2^{26}$ devuelve 195 313, separadas por 2 segundos (Tabla 3). Las dos muestran el mismo ciclo diario: en los 6 días completos del rango, el máximo diario va de 11,7 a 18,3 °C y el mínimo diario de −0,8 a 2,2 °C (con $N = 2^{26}$). El promedio de la semana cambia poco entre ambos valores de N (8,2 y 7,9 °C). Todos los pares de los resultados con N menor aparecen en el de $N = 2^{26}$, porque los prefijos están contenidos unos en otros. Con $N = 2^{16}$ se pierden las variaciones que duran menos que la separación entre mediciones, que con $N = 2^{26}$ se ven como una curva casi continua.

3.7 Bonus: elección de α

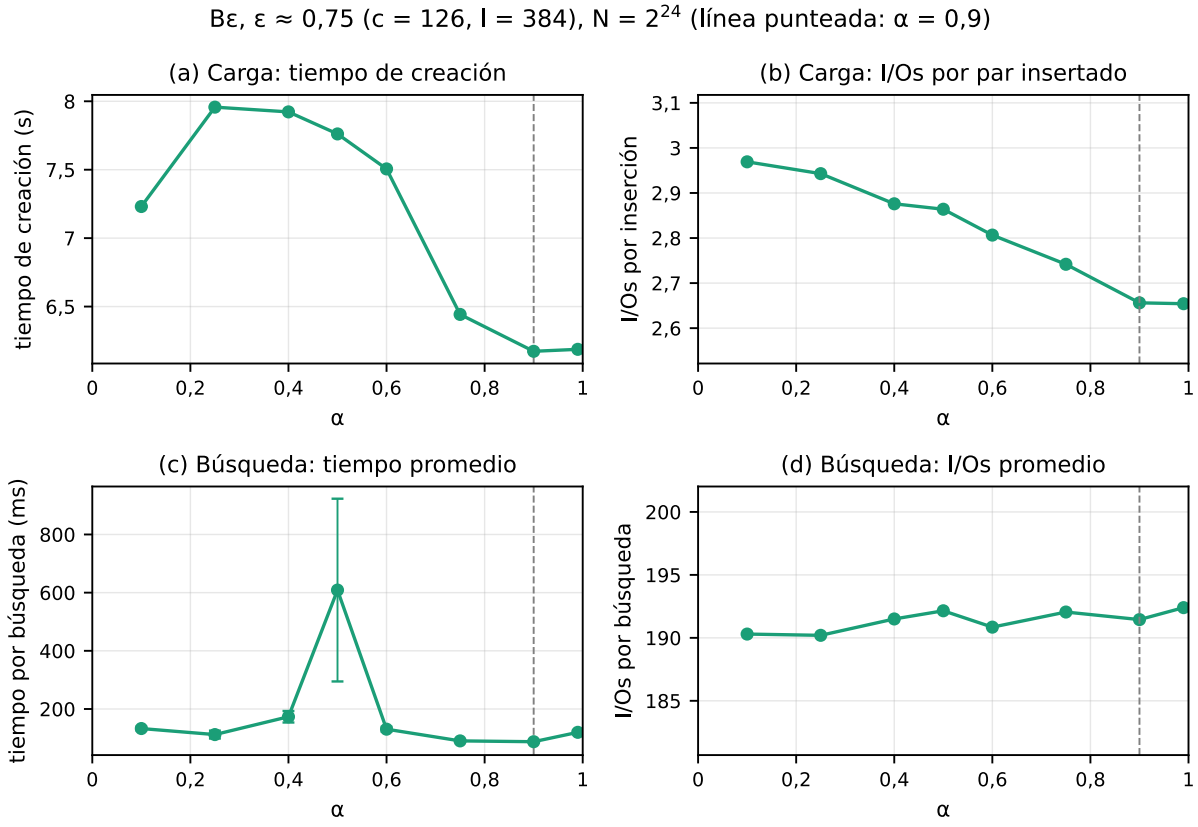


Figura 6: Experimento de α con el B^ε $c = 126$ y $N = 2^{24}$. (a) Tiempo de creación; (b) I/Os por inserción; (c) tiempo por búsqueda: promedio de 20 búsquedas, barras de ± un error estándar; (d) I/Os por búsqueda. Las escalas de (b) y (d) van de 5 % bajo el mínimo a 5 % sobre el máximo.

α	*t* creación (s)	desv. (s)	I/Os por par	Vaciados	Pares por lote	% pares en internos	Nodos	*t* búsq. (ms)	err. est. (ms)	I/Os búsq.
0,1	7,23	0,00	2,969	130 648	5,9	0,91	63 374	132,8	6,4	190,3
0,25	7,96	0,00	2,943	131 095	6,0	0,85	63 402	112,0	13,5	190,2
0,4	7,92	0,00	2,876	133 630	6,5	0,91	63 393	173,3	19,8	191,5
0,5	7,76	0,00	2,864	134 741	6,6	0,91	63 405	608,8	314,1	192,2
0,6	7,51	0,00	2,807	142 611	7,0	0,99	63 292	130,6	8,2	190,8
0,75	6,44	0,00	2,742	154 777	7,7	1,01	63 466	90,3	3,8	192,1
0,9	6,17	0,00	2,656	186 200	8,6	1,15	63 361	87,5	4,0	191,4
0,99	6,19	0,00	2,654	187 999	8,7	1,13	63 336	119,8	6,3	192,4

Tabla 4: Experimento de α con el B^ε $c = 126$ y $N = 2^{24}$. *desv.* es la desviación estándar del tiempo de creación (0 con una sola construcción); el tiempo de búsqueda es el promedio de las 20 búsquedas y *err. est.* su error estándar.

En el B^ε con $c = 126$ (Figura 6, Tabla 4), las I/Os de carga pasan de 2,969 por inserción con el menor α a 2,654 con el mayor (mínimo: 2,654, con $\alpha = 0{,}99$), una baja de 10,6 % (33 % sin contar la raíz), monótona en α. El lote promedio pasa de 5,9 a 8,7 pares, los vaciados de 130 648 a 187 999 y la fracción de pares en nodos internos de 0,91 % a 1,13 %. El tiempo de creación queda entre 6,17 y 7,96 s (el menor con $\alpha = 0{,}9$). Las I/Os de búsqueda quedan entre 190,2 y 192,4 (1,2 % de diferencia) y el tiempo de búsqueda entre 87,5 y 608,8 ms.

Elección. Se escogió $\alpha = 0{,}9$, que es el que usan ambos B^ε en el experimento principal. Sus I/Os de carga (2,656 por par) están 0,08 % sobre el mínimo y sus I/Os de búsqueda, 0,6 % sobre el mínimo. Además tuvo el menor tiempo de creación (6,17 s). Un α más cercano a 1 haría más vaciados (187 999 con $\alpha = 0{,}99$ contra 186 200), es decir, más trabajo de CPU, sin una ganancia apreciable en I/Os. Las I/Os de búsqueda cambian a lo más 1,2 % entre todos los α, así que en este experimento una mejor carga no empeoró la búsqueda. La Sección 4.3 explica estos resultados.

4 Análisis

4.1 Costo amortizado de la inserción

Árbol B. Cada inserción lee los h nodos del camino y escribe la hoja. Un split escribe 3 bloques y ocurre en promedio una vez cada ≈ 240 inserciones en una hoja (cada split crea una hoja nueva, y las hojas tienen en promedio 240 pares), lo que suma unas $3/240$ escrituras por inserción: con $N = 2^{26}$ el árbol B hace 1,012 escrituras por par (exactamente $1 + N + 3 \cdot \text{splits}$ escrituras en total). Por lo tanto, las I/Os por inserción son $h + 1$ más esa pequeña fracción, lo que explica los escalones de la Figura 2 b: suben un poco más de 1 cada vez que el árbol gana un nivel (en 2^{17} y 2^{25}). Cuando la altura sube durante la construcción, las primeras inserciones se hicieron con un nivel menos, así que el promedio queda entre dos escalones.

Árbol B^ε. Cada inserción cuesta 2 I/Os en la raíz. El resto son lotes: cada uno cuesta 2 I/Os (leer y escribir el hijo), más una escritura por nodo nuevo. Las lecturas son exactamente $N + \text{lotes}$ y las escrituras $N + \text{lotes} + \text{nodos}$, y los nodos son pocos (0,0030 y 0,0038 por inserción con $N = 2^{26}$). Si $\overline{m}$ es el tamaño promedio de un lote y $\overline{d}$ el número promedio de niveles que baja cada par,

$$\text{I/Os por inserción} \approx 2 + \frac{2\overline{d}}{\overline{m}}. \tag{1}$$

Con los contadores de la Tabla 2, la Ecuación 1 da $2 + 2 \cdot 8{,}83 \;/\; 158{,}0 = 2{,}112$ para $c = 4$ y $2 + 2 \cdot 2{,}95 \;/\; 8{,}4 = 2{,}702$ para $c = 126$, a 0,003 y 0,005 de lo medido (2,115 y 2,707). Esa diferencia son las escrituras de los nodos nuevos, que la ecuación no incluye. La Figura 2 c muestra que ese término explica las I/Os sin la raíz para todo N.

El análisis amortizado acota ambos factores. Un par baja a lo más $h - 1 = O(\log_{B^\varepsilon} N) = O(\log_B N/\varepsilon)$ niveles. Y cada lote mueve muchos pares si el buffer está casi lleno al vaciarse, lo que se cumple siempre en la raíz (se vacía con $l_{\max}$ pares) y en los demás nodos cuando se vacían porque no les cabe un lote: con $\approx l_{\max}$ pares repartidos entre a lo más $c + 1 \approx B^\varepsilon$ hijos, el hijo más poblado recibe al menos $f \approx l_{\max}/(c+1) = \Omega(B^{1-\varepsilon})$ pares, y cada hijo elegido, al menos αf. Con un argumento de potencial, en el que cada par en un buffer guarda crédito para sus bajadas futuras y cada bajada cuesta $O(1/(\alpha B^{1-\varepsilon}))$ I/Os, el costo amortizado de los vaciados es

$$O\!\left(\frac{h}{\alpha B^{1-\varepsilon}}\right) = O\!\left(\frac{\log_B N}{\alpha\varepsilon B^{1-\varepsilon}}\right) \text{ I/Os por inserción.} \tag{2}$$

En los datos, con $\varepsilon \approx 0{,}25$ los lotes son grandes ($\overline{m} = 158{,}0$, del orden de $506/\ 3{,}15 \approx 161$, porque los nodos internos tienen en promedio 3,15 hijos): aunque cada par baja 8,83 niveles, cada bajada cuesta solo 0,01 I/Os. Con $\varepsilon \approx 0{,}75$ los lotes son de 8,4 pares: cada par baja solo 2,95 niveles, pero cada bajada cuesta 0,24 I/Os. La teoría predice que el costo del árbol B dividido por el de un B^ε (sin la raíz) es $\Theta(\varepsilon B^{1-\varepsilon})$: ≈ 27 para $\varepsilon = 0{,}25$ y $\approx 3{,}6$ para $\varepsilon = 0{,}75$. Lo medido con $N = 2^{26}$ es 32,3 y 5,3. Como $\overline{d}$ crece con la altura, el costo sin la raíz de los B^ε crece lentamente con N (de 0,033 a 0,115 con $c = 4$, mientras la altura pasa de 5 a 11).

La raíz domina. En el modelo de la tarea la raíz se lee y se escribe en cada inserción, y esas 2 I/Os son el 95 % de las I/Os del B^ε $c = 4$ y el 74 % de las del B^ε $c = 126$. Por eso, contando todo, la ventaja de los B^ε sobre el árbol B es solo de 2,2 y 1,7 veces. En un sistema real la raíz (y probablemente los primeros niveles) estaría siempre en RAM, y la diferencia que importa es la de la Figura 2 c. Como ilustración, si cada I/O costara lo que costó en promedio una I/O de búsqueda con $N = 2^{26}$ (0,36 ms por bloque, Tabla 2), insertar los 2^{26} pares directamente en disco tomaría unas 31,9 horas con el árbol B (25,1 horas con la raíz en RAM), mientras que con la raíz en RAM tomaría unas 4,8 horas con el B^ε $c = 126$ y 47 minutos con el B^ε $c = 4$. Es una extrapolación gruesa, porque ignora la caché del sistema operativo y que las escrituras se pueden agrupar.

Tiempo. En la simulación, el tiempo de creación es aproximadamente las I/Os por el costo de copiar un bloque de 4 KiB: 0,08–0,14 µs por I/O con $N \le 2^{20}$, cuando los árboles (≤ 17 MiB) caben en buena parte en las cachés, y 0,12–0,17 µs con $N = 2^{26}$, cuando ocupan 0,78–1,07 GiB y es probable que leer una hoja al azar falle en las cachés y

en la TLB (esto no se midió directamente). El B^ε $c = 4$ es el más barato por I/O (0,12 µs contra 0,17 µs del árbol B) porque casi todas sus I/Os son la raíz, que se usa en cada inserción y por eso se mantiene en la caché. Por eso la ventaja en tiempo puede ser mayor que en I/Os: con $N = 2^{26}$, el árbol B tarda 3,3 veces lo que el B^ε $c = 4$ y hace 2,2 veces sus I/Os.

4.2 Altura en función de ε y costo de búsqueda

Altura. En un árbol con H hojas cuyos nodos internos tienen en promedio F hijos, $h \approx 1 + \log_F H$. Con $N = 2^{26}$, F es 268 en el árbol B, 82 en el B^ε $c = 126$ y 3,15 en el B^ε $c = 4$. Eso da $\log_F H$ = 2,2, 2,8 y 10,3, del orden de los 3, 3 y 10 niveles internos medidos (alturas 4, 4 y 11). La fórmula es aproximada porque supone que todos los niveles tienen la misma cantidad de hijos, y cerca de la raíz no es así (la raíz puede tener pocos hijos). La razón entre los niveles internos de los dos B^ε se compara con la razón 3 entre sus $1/\varepsilon$, que predice $h = O(\log_B N/\varepsilon)$. Con $F \approx 3$, el B^ε $c = 4$ debería sumar un nivel cada vez que N se multiplica por ≈ 3; en los datos sumó 6 niveles entre 2^{15} y 2^{26}, uno por cada factor de $\approx 3{,}6$. Los otros dos suman un nivel cada vez que N se multiplica por un factor del orden de su F.

Búsqueda. Un rango que cubre una fracción p del espacio de llaves intersecta $\approx p \cdot n_\ell + 1$ de los n_ℓ nodos de cada nivel ℓ, así que

$$\text{I/Os por búsqueda} \approx p \cdot n + h, \tag{3}$$

donde n es el total de nodos. Con $p = 604\,800/207\,705\,600 = 0{,}29\ \%$, la Ecuación 3 queda a menos de 1,7 % de lo medido en los N de la Tabla 5. Esto explica la Figura 4 c:

- Con N chico, $p \cdot n$ es menor que 1 (a lo más 0,38 con $N = 2^{15}$) y manda la altura: con $N = 2^{15}$, el B^ε $c = 4$ ($h = 5$) hace 2,17 veces las I/Os del árbol B ($h = 2$).
- Con N grande manda $p \cdot n$ (con $N = 2^{26}$, de 594 a 814, contra alturas de 4 a 11), y el orden lo da el tamaño. El B^ε $c = 4$ tiene 27 % menos nodos que el árbol B porque sus hojas guardan 338 pares en promedio contra 240, y porque guarda el 29,9 % de los pares en buffers de nodos internos, que en el árbol B solo sirven para dirigir la búsqueda. Las curvas se cruzan cerca de $N = 2^{22}$, cuando la diferencia de tamaño compensa la diferencia de altura.

Por lo tanto, para estos rangos el árbol más bajo no es necesariamente el más rápido, sino el más compacto. En una búsqueda puntual el costo sería solo la altura (4, 4 y 11 I/Os con $N = 2^{26}$), y el árbol B y el B^ε $c = 126$ serían más baratos que el B^ε $c = 4$. El tiempo de búsqueda es casi exactamente las I/Os por la latencia del disco: entre 0,02 y 0,66 ms por I/O en todos los N y árboles, así que revisar los buffers de los nodos visitados no tiene un costo de CPU apreciable.

N	Árbol	Nodos n	Altura h	$p \cdot n + h$	I/Os medidas	Diferencia (%)
2^{20}	B	4 287	3	15,5	15,5	0,2
2^{20}	B^ε c=4	3 164	8	17,2	17,1	-0,9
2^{20}	B^ε c=126	3 979	3	14,6	14,3	-1,7
2^{23}	B	34 675	3	104,0	104,6	0,6
2^{23}	B^ε c=4	25 425	9	83,0	82,6	-0,5
2^{23}	B^ε c=126	31 650	4	96,2	95,9	-0,2
2^{26}	B	279 471	4	817,8	822,4	0,6
2^{26}	B^ε c=4	204 093	11	605,3	601,5	-0,6
2^{26}	B^ε c=126	253 328	4	741,6	740,7	-0,1

Tabla 5: I/Os por búsqueda: modelo de la Ecuación 3 y valor medido (promedio de todas las búsquedas).

4.3 Efecto de α

Un lote cuesta 2 I/Os sin importar cuántos pares lleve, así que bajar grupos chicos es lo que desperdicia I/Os: si se dejan en el buffer, bajan más tarde junto con los pares que lleguen después. El umbral $\lceil \alpha f \rceil$ decide cuáles grupos bajan. Un α alto hace más vaciados, porque cada uno libera menos espacio, pero en esta implementación un vaciado no cuesta I/Os propias: la raíz se lee en cada inserción y un nodo interno se vacía cuando ya se leyó para recibir un

lote. Solo cuesta CPU (agrupar el buffer). Por eso las I/Os de carga tienden a bajar al subir α, y con $c = 126$ bajaron de forma monótona.

El efecto depende de cuántos pares le tocan a cada hijo. Con $c = 4$, un vaciado reparte ≈ 500 pares entre unos 3,15 hijos, así que los grupos tienen más de cien pares y los lotes ya son grandes (158,0 pares en promedio con $\alpha = 0{,}9$, Tabla 2): se espera que ahí α ahorre poco, aunque no se midió. Con $c = 126$, en cambio, $f \approx 384/82 \approx 4{,}7$: con tan pocos pares por hijo, es esperable que el tamaño de los grupos varíe mucho de un hijo a otro. Con $\alpha = 0{,}1$ el umbral es 1 y baja casi todo grupo no vacío; con $\alpha = 0{,}9$ el umbral es 5 y los grupos chicos esperan. Ahí α cambia el costo en un 33 % sin contar la raíz.

En la búsqueda, α cambia cuántos pares quedan en buffers internos (de 0,91 % a 1,13 % con $c = 126$), pero esos nodos se leen igual, porque están en el camino de la búsqueda. Según la Ecuación 3 lo que importa es el número de nodos, que con α cambia a lo más 0,3 %, y las I/Os de búsqueda cambian a lo más 1,2 %. El tiempo por I/O de búsqueda varía entre 0,46 y 3,17 ms entre valores de α, así que las diferencias de tiempo de búsqueda entre α vienen sobre todo de la variación de la latencia del disco. Así, el compromiso que menciona el enunciado (mejor carga a cambio de peor búsqueda) no aparece en este experimento: la búsqueda casi no depende de α, y el criterio para elegirlo es la carga.

5 Conclusión

Se implementaron un árbol B y dos árboles B^ε con nodos de 4096 bytes. Se construyeron sobre un disco simulado que cuenta cada transferencia de bloque, se guardaron en un archivo y se buscaron leyendo de él con `O_DIRECT`, para $N = 2^{15}, ..., 2^{26}$ pares reales de temperatura. Todas las búsquedas se verificaron contra fuerza bruta y las invariantes de las estructuras se probaron por separado.

Hipótesis.

1. *Inserción: se cumplió en I/Os y en parte en tiempo.* En I/Os el orden B^ε $c = 4$ < B^ε $c = 126$ < B se cumplió para todo N, y en tiempo en 11 de los 12: con $N = 2^{26}$, 2,115, 2,707 y 4,712 I/Os por inserción, y 16,6, 27,2 y 54,2 s. Las I/Os de los B^ε están dominadas por las 2 de la raíz; sin ellas, la diferencia con el árbol B es de 32,3 y 5,3 veces, del orden de $\varepsilon B^{1-\varepsilon}$ que predice el análisis amortizado. La causa es el tamaño de los lotes: 158,0 pares por lote con $\varepsilon \approx 0{,}25$ contra 8,4 con 0,75.
2. *Altura y búsqueda: se cumplió en parte.* Las alturas fueron las esperadas (11 contra 4 y 4 con $N = 2^{26}$), y con N chico el B^ε $c = 4$ fue el peor en búsquedas (2,17 veces las I/Os del árbol B con 2^{15}). Pero desde $N = 2^{22}$ fue el que menos leyó, no solo similar a los otros: con $N = 2^{26}$ hizo 0,73 veces las I/Os del árbol B. No se anticipó que una búsqueda que devuelve cientos de bloques cuesta $p \cdot n + h$, y que los buffers del B^ε también guardan datos, lo que lo hace el árbol más compacto.
3. *Tamaño: se cumplió.* Con $N = 2^{26}$, el B^ε $c = 4$ usó 27 % menos bloques que el árbol B y el de $c = 126$, 9 % menos.

En el bonus, el costo de carga del B^ε $c = 126$ baja al subir α (de forma monótona), mientras que la búsqueda casi no depende de α. Por eso se usó $\alpha = 0{,}9$ en ambos B^ε.

Limitaciones y mejoras.

- El modelo cuenta 2 I/Os de raíz por inserción, lo que esconde la mayor parte de la ventaja de los B^ε. Una versión futura podría mantener la raíz y los primeros niveles en un *buffer pool* (por ejemplo, LRU) y contar solo los bloques que salen de él, lo que se parecería más a un sistema real.
- Las inserciones se midieron en RAM. Medir algunas construcciones directamente en disco permitiría validar la extrapolación de la Sección 4.
- Solo se usaron rangos de una semana. Variar el ancho del rango (desde búsquedas puntuales hasta meses) mostraría en qué punto conviene cada árbol, según la Ecuación 3.
- Los tiempos dependen de la máquina (Tabla 1) y de la carga que tenía al medir, y cada construcción se midió una sola vez; repetir las construcciones y usar la mediana daría tiempos más robustos. Las I/Os, el tamaño y la altura no dependen de la máquina.

- No se implementó el borrado (no se pedía), ni la carga masiva de datos ordenados, que permitiría construir árboles completamente llenos.

Anexo: resultados completos del experimento principal

N	Árbol	*t* creación (s)	I/Os creación (millones)	I/Os por par	Nodos	MiB	Altura	*t* búsqueda (ms)	I/Os búsqueda	Resultados
2^{15}	B	0,009	0,10	3,002	132	0,5	2	0,06	2,4	97
	B^ε c=4	0,006	0,07	2,033	97	0,4	5	0,11	5,2	97
	B^ε c=126	0,006	0,07	2,221	124	0,5	2	0,11	2,5	97
2^{16}	B	0,017	0,20	3,007	263	1,0	2	0,30	2,8	191
	B^ε c=4	0,011	0,13	2,040	197	0,8	5	0,27	5,4	191
	B^ε c=126	0,013	0,15	2,303	251	1,0	3	0,23	3,8	191
2^{17}	B	0,040	0,44	3,389	531	2,1	3	0,57	4,6	380
	B^ε c=4	0,023	0,27	2,047	393	1,5	6	0,73	6,9	380
	B^ε c=126	0,028	0,31	2,351	492	1,9	3	0,50	4,4	380
2^{18}	B	0,089	0,97	3,700	1 062	4,1	3	0,92	6,1	761
	B^ε c=4	0,048	0,54	2,055	787	3,1	6	1,36	8,3	761
	B^ε c=126	0,058	0,62	2,368	992	3,9	3	1,11	6,0	761
2^{19}	B	0,202	2,02	3,856	2 139	8,4	3	2,23	9,2	1 525
	B^ε c=4	0,101	1,08	2,062	1 569	6,1	7	2,33	11,5	1 525
	B^ε c=126	0,137	1,25	2,385	1 991	7,8	3	1,54	8,7	1 525
2^{20}	B	0,583	4,13	3,934	4 287	16,7	3	4,06	15,5	3 049
	B^ε c=4	0,215	2,17	2,070	3 164	12,4	8	4,39	17,1	3 049
	B^ε c=126	0,280	2,54	2,424	3 979	15,5	3	4,40	14,3	3 049
2^{21}	B	1,201	8,33	3,973	8 601	33,6	3	18,51	28,2	6 086
	B^ε c=4	0,490	4,36	2,078	6 351	24,8	8	7,76	26,4	6 086
	B^ε c=126	0,637	5,24	2,497	7 930	31,0	3	8,12	26,1	6 086
2^{22}	B	2,611	16,75	3,993	17 266	67,4	3	15,21	53,9	12 207
	B^ε c=4	0,951	8,74	2,085	12 637	49,4	9	18,34	45,4	12 207
	B^ε c=126	1,431	10,84	2,586	15 843	61,9	4	14,80	49,9	12 207
2^{23}	B	5,594	33,58	4,003	34 675	135,4	3	45,20	104,6	24 448
	B^ε c=4	1,880	17,55	2,093	25 425	99,3	9	30,88	82,6	24 448
	B^ε c=126	2,960	22,13	2,638	31 650	123,6	4	40,50	95,9	24 448
2^{24}	B	11,029	67,24	4,008	69 581	271,8	3	80,10	207,1	48 865
	B^ε c=4	3,846	35,23	2,100	50 907	198,9	10	87,48	157,5	48 865
	B^ε c=126	6,047	44,56	2,656	63 361	247,5	4	84,48	188,0	48 865
2^{25}	B	24,792	148,03	4,412	139 394	544,5	4	149,12	411,8	97 706
	B^ε c=4	8,074	70,69	2,107	101 764	397,5	10	157,28	304,3	97 706
	B^ε c=126	13,111	89,68	2,673	126 696	494,9	4	134,34	372,2	97 706
2^{26}	B	54,182	316,22	4,712	279 471	1 091,7	4	252,15	822,4	195 420
	B^ε c=4	16,581	141,92	2,115	204 093	797,2	11	242,24	601,5	195 420
	B^ε c=126	27,249	181,67	2,707	253 328	989,6	4	278,77	740,7	195 420

Tabla 6: Resultados del experimento principal. El tiempo, las I/Os y los resultados de búsqueda son promedios de las 50 búsquedas. Las I/Os de creación son las del disco simulado y las de búsqueda son bloques leídos del archivo.